Multidimensional Data

Lecture 26

CS 1001

Kirk Duran

Today

- Extend the sequence model from one dimension to data organized by multiple coordinates.
- Represent two-dimensional data using nested Python lists.
- Read and mutate elements using row and column indexes.
- Trace nested iteration over rows and individual elements.
- Generalize the same model to three and more dimensions.
- Introduce NumPy arrays as a regular multidimensional representation for numerical data.

Key idea

Multidimensional data does not require a new notion of indexing. It requires applying familiar indexing and iteration at more than one structural level.

The Boundary for This Lecture

Today

- nested lists
- rows, columns, and coordinates
- two-stage indexing
- mutation and aliasing in nested lists
- nested iteration
- 3D and n-dimensional structure
- NumPy arrays, shape, slicing, and reshape

Not today

- general-purpose built-in function catalogs
- searching or sorting algorithms
- algorithmic complexity
- advanced NumPy broadcasting rules
- functions over structured data
- object-oriented representations

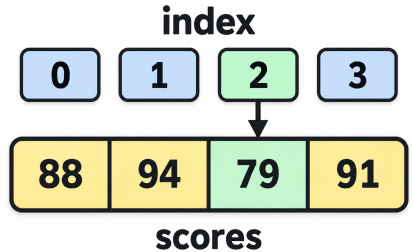
Part 1: From One Dimension to Multiple Dimensions

A One-Dimensional Sequence Uses One Position

One-dimensional data

```
scores = [88, 94, 79, 91]
```

- One integer index identifies a position.
- `scores[2]` selects one element.
- The structure has one natural positional axis: left to right.



Two Coordinates Identify a Cell

Seats, pixels, and board squares are naturally identified by two coordinates.

	Column 0	Column 1	Column 2
Row 0	4	7	1
Row 1	8	5	2
Row 2	6	3	9

Definition

Multidimensional data is data whose organization requires more than one positional coordinate to identify a particular component.

Key idea

Two coordinates identify a cell: row first, then column.

A Python 2D List Is a List Whose Elements Are Lists

Nested list

```
grid = [[4, 7, 1],
        [8, 5, 2],
        [6, 3, 9]]
```

- The outer list contains three elements.
- Each outer element is itself a list.
- We interpret each inner list as one row.
- This creates a useful two-dimensional *conceptual* structure from ordinary lists.

Key idea

Python does not introduce a special “2D list” type. The representation is nested ordinary lists.

Rectangular and Ragged Nested Lists

Rectangular

```
[[1, 2, 3],
  [4, 5, 6],
  [7, 8, 9]]
```

Every row has the same number of elements.

Ragged

```
[[1, 2],
  [3, 4, 5],
  [6]]
```

Different rows may have different lengths.

Consequence

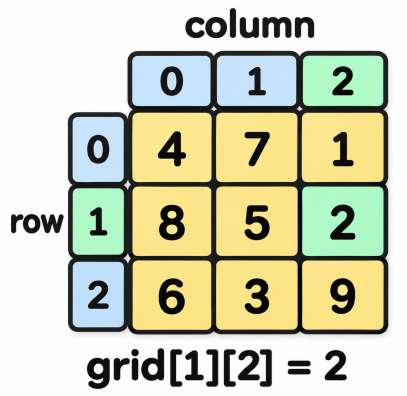
A nested list can represent a grid, but Python does not require it to be rectangular.

The Row–Column Coordinate Model

For a rectangular nested list, we use the convention:

```
grid[row][column]
```

- first index: which row?
- second index: which element inside that row?
- both indexes remain zero-based



Part 2: Indexing, Mutation, and References

Two-Dimensional Indexing Is Two Ordinary Indexing Operations

Suppose:

Data

```
grid = [[4, 7, 1], [8, 5, 2], [6, 3, 9]]
```

Evaluate:

```
grid[1][2]
```

1. `grid[1]` evaluates to the row `[8, 5, 2]`.
2. The remaining `[2]` indexes that row.
3. The final value is 2.

Trace a Two-Index Expression

Expression

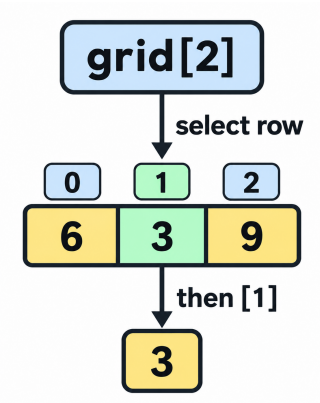
`grid[2][1]`

`grid[2]` $\rightarrow$ `[6, 3, 9]`

`[6, 3, 9][1]` $\rightarrow$ `3`

Key idea

Bracket operations associate left to right: obtain the inner list first, then index that list.



One Index Selects an Entire Row

Example

```
grid = [[4, 7, 1], [8, 5, 2], [6, 3, 9]]
row = grid[1]
```

Afterwards:

$$\texttt{row} \rightarrow \texttt{[8, 5, 2]}$$

- The outer list stores references to its element values.
- Here, the selected element value is itself a list.
- `row` and `grid[1]` therefore refer to the same inner list.

Mutating One Cell

Before

```
grid = [[4, 7, 1], [8, 5, 2], [6, 3, 9]]
```

Mutation

```
grid[1][2] = 20
```

After

```
[[4, 7, 1], [8, 5, 20], [6, 3, 9]]
```

Key idea

Indexed assignment mutates the selected inner list. The surrounding outer list still refers to that same row list.

The Reference Model Becomes More Important When Lists Are Nested

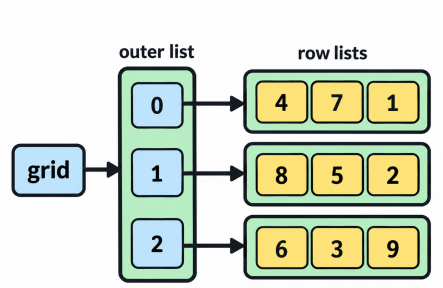
Conceptually:

`grid` ⟶ outer list

outer positions ⟶ row lists

row positions ⟶ element values

- The rows are separate list values only if they were created separately.
- Multiple outer positions can refer to the same row list.



Aliasing Can Occur Inside a Nested List

One row, referenced three times

```
row = [0, 0, 0]
grid = [row, row, row]

grid[0][1] = 7
```

What is `grid` afterward?

```
[[0, 7, 0], [0, 7, 0], [0, 7, 0]]
```

Why?

The outer list contains three references to the same inner list. The mutation changes that one shared row.

To make rows independent, create separate inner lists, for example `[[0, 0, 0], [0, 0, 0], [0, 0, 0]]`.

Row Slicing Uses Ordinary List Slicing

Data

```
grid = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

Select rows 1 through the end

```
rows = grid[1:]
```

The new outer list contains references to:

```
[[4, 5, 6], [7, 8, 9]]
```

Reference detail

Slicing creates a new outer list, but the selected inner row lists are still shared with `grid`.

Native nested lists have no special column slice such as `grid[:, 1]`; a column must be selected from the rows during iteration. NumPy will provide direct multidimensional slicing later.

Part 3: Nested Iteration

The Outer Loop Naturally Visits Rows

Code

```
grid = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]

for row in grid:
    ...
```

Successive bindings of `row` are:

$$\texttt{[1, 2, 3]} \rightarrow \texttt{[4, 5, 6]} \rightarrow \texttt{[7, 8, 9]}$$

Key idea

Iterating over the outer list visits its elements. Because those elements are row lists, each loop iteration receives one row.

A Nested Loop Visits Individual Cells

Code

```
for row in grid:
   for value in row:
      ...
```

outer loop $\rightarrow$ selects one row

inner loop $\rightarrow$ visits each value in that row

Key idea

The inner loop completes for the current row before the outer loop advances to the next row.

Trace Nested Iteration

For `grid = [[1, 2], [3, 4]]`:

Step	row	value
1	`[1, 2]`	1
2	`[1, 2]`	2
3	`[3, 4]`	3
4	`[3, 4]`	4

Control-flow fact

The outer binding remains the same while the inner loop advances through the current row.

Index-Based Nested Iteration Exposes Coordinates

For a known 3×3 grid:

Code

```
for row in range(3):
   for col in range(3):
      value = grid[row][col]
```

- `row` identifies the outer-list position.
- `col` identifies the position inside that row.
- The pair `(row, col)` identifies one cell.

Key idea

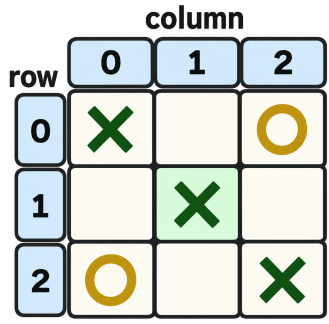
Direct iteration emphasizes values; index-based iteration is useful when the coordinates themselves matter.

Example Representation: A Tic-Tac-Toe Board

Board

```
board = [["X", "", "O"],
         ["", "X", ""],
         ["O", "", "X"]]
```

- row and column coordinates identify squares;
- an empty string can represent an unoccupied square;
- mutation can represent a move.



Key idea

The data representation is the topic here; game-winning logic and algorithms are outside today's scope.

Part 4: Three and More Dimensions

A Third Dimension Adds Another Structural Level

A three-dimensional nested list can be interpreted as a sequence of two-dimensional layers.

layer $\rightarrow$ **row** $\rightarrow$ **column** $\rightarrow$ **value**

Suppose:

Data

```
cube = [[[1, 2], [3, 4]],
        [[5, 6], [7, 8]]]
```

Evaluate `cube[1][0][1]`:

1. `cube[1]` $\rightarrow$ `[[5, 6], [7, 8]]`
2. `[0]` $\rightarrow$ `[5, 6]`
3. `[1]` $\rightarrow$ `6`

Key idea

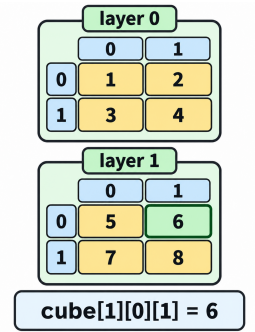
Additional dimensions repeat the same rule: each index moves one structural level deeper.

Nested Iteration Mirrors Structural Depth

Three levels

```
for layer in cube:
   for row in layer:
      for value in row:
         ...
```

Each loop operates on the sequence value produced by the level above it.



Key idea

Each additional nested sequence level can require another coordinate and another level of iteration. This same reasoning generalizes beyond three dimensions.

Nested Lists Are Flexible—Sometimes Too Flexible

Python nested lists permit:

- rows of different lengths;
- inner lists that are shared through aliases;
- elements of different types;
- arbitrary nesting depth.

That flexibility is useful for general programming, but regular numerical data often benefits from a structure with explicit dimensional shape.

Key idea

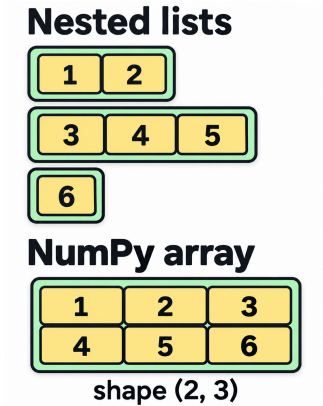
This is the point where NumPy provides a different abstraction: the multidimensional array.

Part 5: NumPy Multidimensional Arrays

Why Introduce NumPy Here?

NumPy is designed around regular numerical arrays.

- dimensional shape is explicit;
- elements use a common numerical data type;
- multidimensional indexing is built into the array abstraction;
- arithmetic can operate over many array elements at once.



Creating a NumPy Array

Import and construction

```
import numpy as np

matrix = np.array([[1, 2, 3],
                   [4, 5, 6]])
```

- `np.array(...)` constructs a NumPy array from the provided data.
- The result is a two-dimensional NumPy `ndarray`.
- The rows and columns are part of the array's explicit shape.

Scope

We use NumPy only to introduce multidimensional array operations needed for this unit; this is not a full NumPy API survey.

Shape Describes the Size of Each Dimension

For:

Array

```
matrix = np.array([[1, 2, 3], [4, 5, 6]])
```

`matrix.shape` $\rightarrow$ `(2, 3)`

Interpret the tuple as:

2 rows $\times$ 3 columns

Key idea

Shape makes the dimensional structure explicit rather than leaving it as an informal interpretation of nested lists.

NumPy Exposes Dimension Count and Element Type

Given

```
matrix = np.array([[1, 2, 3], [4, 5, 6]])
```

Attribute	Meaning
`matrix.ndim`	number of dimensions
`matrix.shape`	size along each dimension
`matrix.dtype`	stored element data type

For this array

`ndim` is 2; `shape` is `(2, 3)`.

NumPy Uses Multidimensional Index Syntax

Nested list

```
grid[1][2]
```

Two ordinary indexing operations.

NumPy array

```
matrix[1, 2]
```

One array indexing operation with two coordinates.

row coordinate first, column coordinate second

NumPy Can Slice Along Individual Dimensions

Suppose `matrix` has shape `(3, 4)`.

Expression	Selection
`matrix[1, :]`	all columns from row 1
`matrix[:, 2]`	column 2 from all rows
`matrix[0:2, 1:3]`	a rectangular subregion

Key idea

Each position in the comma-separated index controls one array dimension.

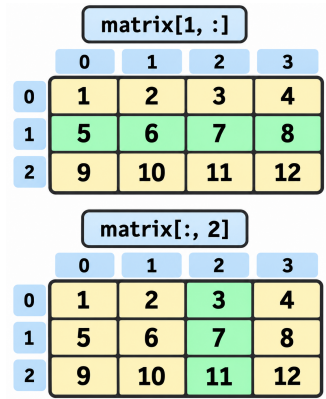
Visualizing Row and Column Slices

Row slice

```
matrix[1, :]
```

Column slice

```
matrix[:, 2]
```



Array Arithmetic Can Express Elementwise Transformation

Array

```
matrix = np.array([[1, 2], [3, 4]])
```

Elementwise operation

```
doubled = matrix * 2
```

`doubled` → `[[2, 4], [6, 8]]`

Key idea

NumPy lets an operation describe a transformation over the array as a whole instead of requiring an explicit nested Python loop for this simple case.

Reshape Changes Dimensional Organization

Start with six values

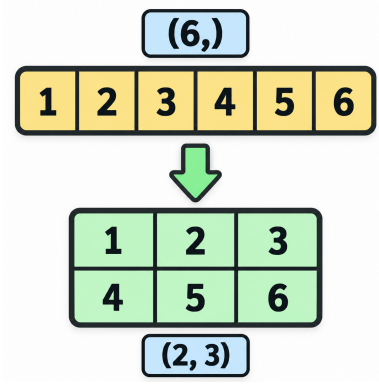
```
data = np.array([1, 2, 3,
    4, 5, 6])
```

Reshape

```
matrix = data.reshape((2, 3))
```

Key idea

The new shape must preserve the number of elements: $6 = 2 \times 3$.



Matrix Multiplication Is Different from Elementwise Multiplication

For compatible two-dimensional arrays:

Matrix product

```
result = a @ b
```

- `a * b` means elementwise multiplication in NumPy.
- `a @ b` means the linear-algebra matrix product.
- Matrix dimensions must be compatible for the product to be defined.

Scope

We use this only to distinguish two common array operations. Matrix-multiplication algorithms are outside this course unit.

Nested Lists and NumPy Arrays Serve Different Roles

Property	Nested lists	NumPy ndarray
Structure	flexible nesting	explicit dimensions
Rectangular required?	no	regular shape
Element types	may vary	common dtype
2D indexing	`a[r][c]`	`a[r, c]`
Column slicing	manual iteration	direct multidimensional slice
Numeric whole-array ops	explicit iteration	array operations

Key idea

The choice of representation should match the structure and operations the program needs.

Integrated Prediction

Consider:

Code

```
matrix = np.array([[2, 4, 6],
                   [8, 10, 12]])

column = matrix[:, 1]
changed = matrix * 2
```

Predict:

1. `matrix.shape`
2. the value selected by `matrix[1, 2]`
3. the values in `column`
4. the values in `changed`
5. whether `matrix * 2` changes the original `matrix`

Multidimensional Data: The Complete Picture

Representation	Core reasoning model
1D list	one index selects one element
nested 2D list	first index selects row, second selects cell
nested 3D list	layer $\rightarrow$ row $\rightarrow$ cell
NumPy array	one coordinate per explicit array dimension

Key idea

More dimensions add coordinates, not a fundamentally different execution model. Trace one structural level at a time.

Takeaways

1. A two-dimensional Python list is a nested list: an outer list whose elements are row lists.
2. `grid[row][col]` is evaluated as two ordinary indexing operations.
3. Nested mutable structures require careful reference reasoning because inner lists can be aliased.
4. Nested loops mirror nested structure: each loop descends one sequence level.
5. Three and more dimensions extend the same coordinate model.
6. NumPy arrays make regular multidimensional shape explicit and support multidimensional indexing, slicing, arithmetic, and reshaping.

Next

Lecture 27 asks how to design and reason about **functions over structured data**.